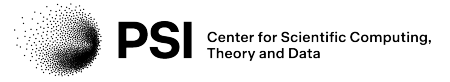


Fast Feedback Indexer



Accelerating Protein Crystallography at SLS

Hans-Christian Stadler

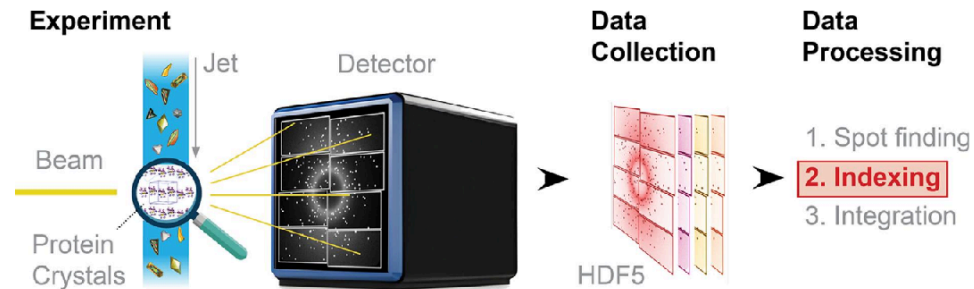
hans-christian.stadler@psi.ch

Paul Scherrer Institut

July 27, 2026

Overview

- Protein crystallography at the Swiss Light Source (SLS)
- The data challenge: high-speed detectors
- Existing algorithm limitations
- The need for fast feedback indexing
- Our approach: GPU-accelerated algorithm
- Algorithm architecture: five-step process
- Results and future work



Motivation: SLS Upgrade

The SLS (Swiss Light Source) at PSI was upgraded to higher luminance

Benefits:

- Coherent monochromatic X-rays
- Study protein crystals in frozen state
- Insights for biological and drug research

Challenge:

- More light means less exposure
- Detectors: 16 megapixel at 1 kHz
- Potential terabytes of data per day
- Storage and analysis are expensive

Solution: Throw away bad data at the source $\Rightarrow$ Need for **fast feedback analysis** to detect bad data

The Performance Problem

Existing solution: C++ indexing algorithm using state-of-the-art library

Performance on one CPU core:

Metric	Result
CPI	0.3988
SP MFLOP/s	5798.4982
Memory bandwidth	143.38 MB/s
Operational intensity	40.44
Vectorization ratio	91.86%

Problem:

- Indexing takes ≈ 80 ms per image
- Target: < 1 ms per image
- Known crystal structure allowed

Conclusion: Sequential algorithm is performant, but not fast enough.

Our Solution: GPU-Accelerated Indexing

New algorithm:

- Leverage GPU parallelism (CUDA/C++)
- Brute force parallel sampling
- Least squares refinement
- Exploit known crystal structure

Implementation:

- C++/CUDA code on GitHub
- Integrated into major crystallography software:
 - Dials (conda-forge package available)
 - CrystFEL
- Used in JungfrauJoch crystallography pipeline, which is able to analyse data on the fly for the first time with sufficient speed

Indexing in a nutshell

Crystals are made up of similar cells, that all reflect light in the same direction, leading to spots on the detector for the coherent monochromatic light of the SLS beam, see [Wikipedia - Bragg's law](#)

These spots can be mapped to a sphere (Ewald sphere). The cell vectors need to have an integer dot product with the spots on the Ewald sphere.

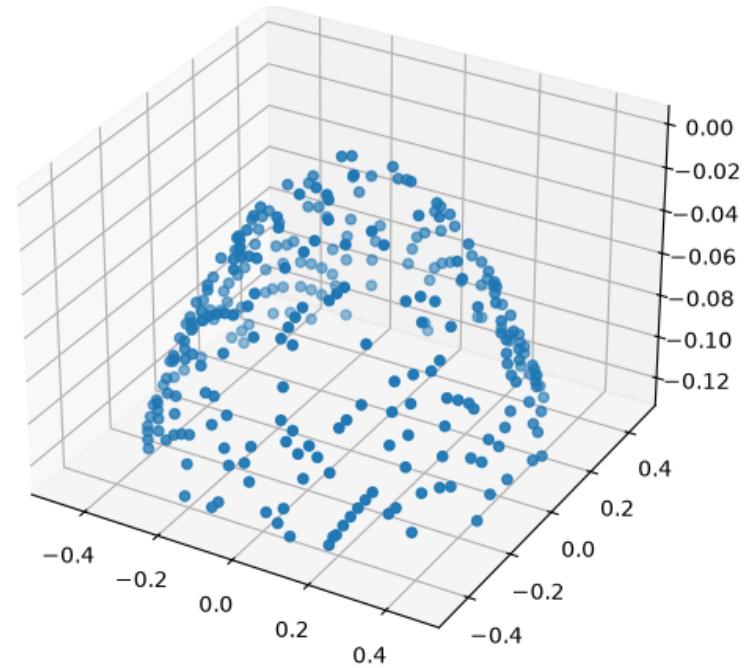


Figure 3: Spots on Ewald sphere

Algorithm: Step 1 — Direction Sampling

Brute-force search for unit cell vectors

- Sample many directions on sphere
- Quality score: number of spots with integer dot product
- Calculations are **independent** → perfect for GPU

Collect best directions: Sorting within blocks, spin locks, merging → not optimal, but effective

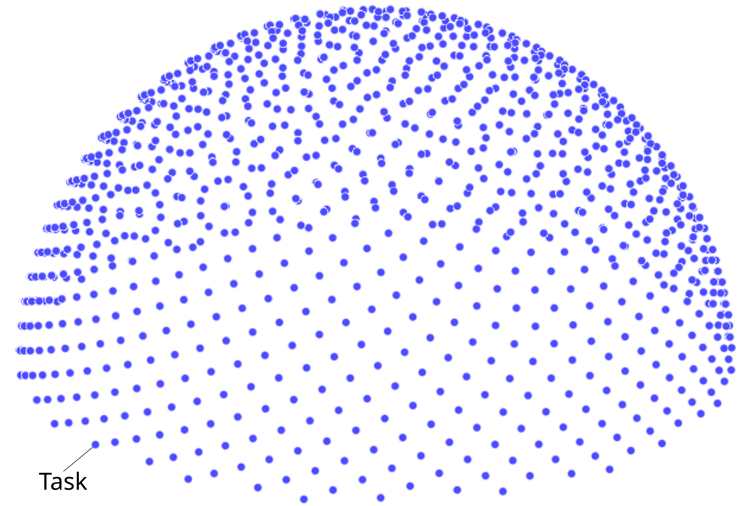


Figure 4: Vector sampling

Algorithm: Step 2 — Vector Refinement

Least-squares refinement of best directions

1. Calculate integer dot product per spot
2. Select spots within threshold error margin
3. Adapt vector using least squares
4. Decrease threshold, repeat

Only a few best directions to refine

→ not well suited for GPU

Algorithm: Step 3 — Rotation Sampling

**Search for optimal rotation
around best vector**

- Attach other two unit cell vectors
- Sample rotation angles
- Score combines all three vectors
- Independent → perfect for GPU

Data collection: Similar to step 1 —
sorting, spin locks, merging → not optimal,
but effective

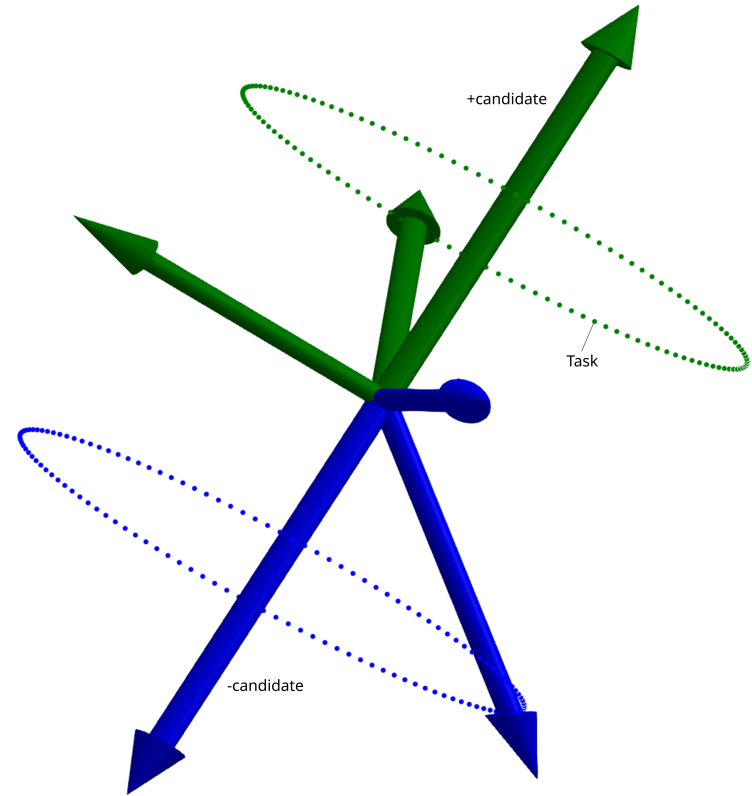


Figure 5: Rotation angle sampling

Algorithm: Steps 4 & 5 — Assembly and Final Cell Refinement

Step 4: Candidate Unit Cell Assembly

- Assemble best cells
- Performed on GPU for consistency
- Too few candidates → not well suited for GPU

Step 5: Final Cell Refinement

- Same refinement as step 2
- Executed on CPU (not well suited for GPU)
- Last step before output

Results and Impact

Performance

- Below 1ms on a single Ampere generation GPU
- Memory usage low enough to keep 3-4 operations in flight

Integration

- Integrated into Dials and CrystFEL
- Indexing is no longer the bottleneck
- Conda-forge package available
- Used in Jungfrauoch crystallography pipeline

Impact

- First reliable fast feedback for bad data rejection as part of Jungfrauoch
- Enables efficient data handling at SLS
- Open source: github.com/paulscherrerinstitute/fast-feedback-indexer

Conclusion and Outlook

Summary

- Algorithm design tailored for GPU acceleration
- 5-step sampling/refinement approach
- Successfully integrated into production software
- Achieved <1 ms indexing throughput

Future Work

- Further optimization of data collection patterns

Thank You!

Hans-Christian Stadler

hans-christian.stadler@psi.ch